

Otimização

3.2.1 Método de pesquisa aleatória

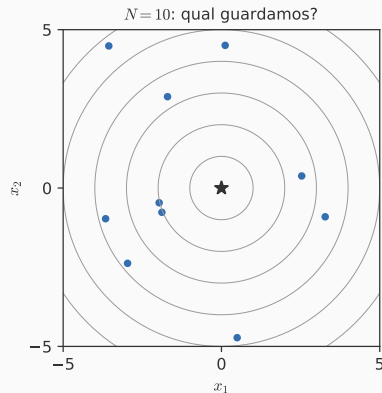
José Firmino Aguilar Madeira

Sem derivadas e sem direção: que ponto guardamos?

O 3.1 terminou com: *se não conhecermos ∇f , como escolhemos para onde andar?* Primeira resposta: **não escolhemos** — experimentamos pontos ao acaso.

$f(x_1, x_2) = x_1^2 + x_2^2$ em $[-5, 5]^2$; geramos 10 pontos uniformes.

Qual destes pontos guardaríamos?



Amostras acumuladas: o melhor até agora
nunca piora.

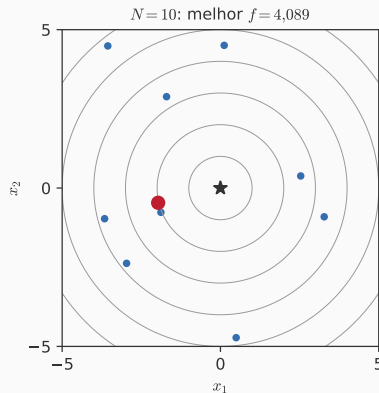
Sem derivadas e sem direção: que ponto guardamos?

O 3.1 terminou com: *se não conhecermos ∇f , como escolhemos para onde andar?* Primeira resposta: **não escolhemos** — experimentamos pontos ao acaso.

$f(x_1, x_2) = x_1^2 + x_2^2$ em $[-5, 5]^2$; geramos 10 pontos uniformes.

Qual destes pontos guardaríamos?

→ o que tiver **menor f** .



Amostras acumuladas: o melhor até agora
nunca piora.

Sem derivadas e sem direção: que ponto guardamos?

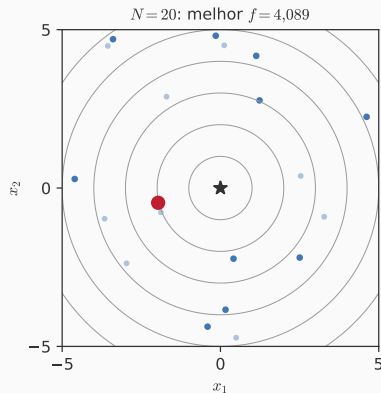
O 3.1 terminou com: *se não conhecermos ∇f , como escolhemos para onde andar?* Primeira resposta: **não escolhemos** — experimentamos pontos ao acaso.

$f(x_1, x_2) = x_1^2 + x_2^2$ em $[-5, 5]^2$; geramos 10 pontos uniformes.

Qual destes pontos guardaríamos?

→ o que tiver **menor f** .

Mais 10 pontos. E agora? Guardamos o melhor de *todos* — aqui nenhum dos novos é melhor: fica o mesmo.



Amostras acumuladas: o melhor até agora nunca piora.

Sem derivadas e sem direção: que ponto guardamos?

O 3.1 terminou com: *se não conhecermos ∇f , como escolhemos para onde andar?* Primeira resposta: **não escolhemos** — experimentamos pontos ao acaso.

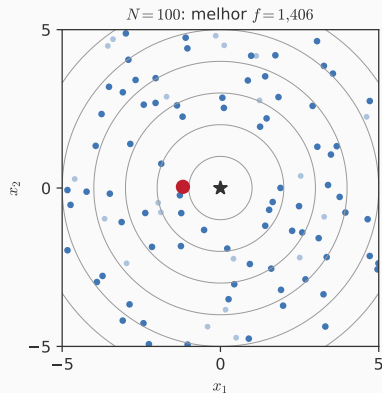
$f(x_1, x_2) = x_1^2 + x_2^2$ em $[-5, 5]^2$; geramos 10 pontos uniformes.

Qual destes pontos guardaríamos?

→ o que tiver **menor f** .

Mais 10 pontos. E agora? Guardamos o melhor de *todos* — aqui nenhum dos novos é melhor: fica o mesmo.

$N = 100$: o melhor aproxima-se ($f = 1,406$).



Amostras acumuladas: o melhor até agora nunca piora.

Sem derivadas e sem direção: que ponto guardamos?

O 3.1 terminou com: *se não conhecermos ∇f , como escolhemos para onde andar?* Primeira resposta: **não escolhemos** — experimentamos pontos ao acaso.

$f(x_1, x_2) = x_1^2 + x_2^2$ em $[-5, 5]^2$; geramos 10 pontos uniformes.

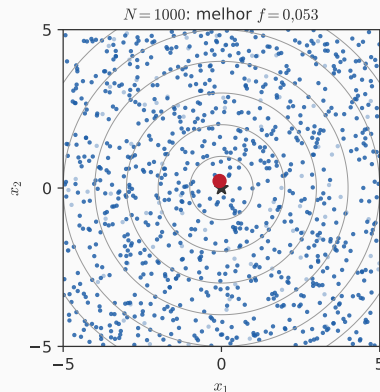
Qual destes pontos guardaríamos?

→ o que tiver **menor f** .

Mais 10 pontos. E agora? Guardamos o melhor de *todos* — aqui nenhum dos novos é melhor: fica o mesmo.

$N = 100$: o melhor aproxima-se ($f = 1,406$).

$N = 1000$: $f = 0,053$, perto do mínimo.



Amostras acumuladas: o melhor até agora
nunca piora.

Sem derivadas e sem direção: que ponto guardamos?

O 3.1 terminou com: *se não conhecermos ∇f , como escolhemos para onde andar?* Primeira resposta: **não escolhemos** — experimentamos pontos ao acaso.

$f(x_1, x_2) = x_1^2 + x_2^2$ em $[-5, 5]^2$; geramos 10 pontos uniformes.

Qual destes pontos guardaríamos?

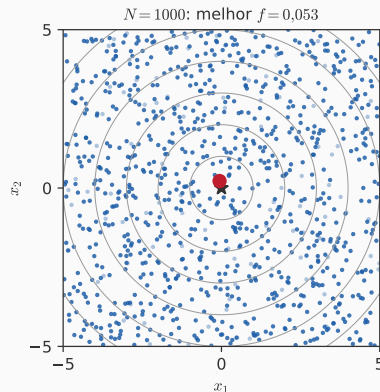
→ o que tiver **menor f** .

Mais 10 pontos. E agora? Guardamos o melhor de *todos* — aqui nenhum dos novos é melhor: fica o mesmo.

$N = 100$: o melhor aproxima-se ($f = 1,406$).

$N = 1000$: $f = 0,053$, perto do mínimo.

gerar → avaliar → guardar o melhor



Amostras acumuladas: o melhor até agora
nunca piora.

Então o algoritmo é quase trivial

Pesquisa aleatória pura: domínio

$D = [a, b]$, N amostras

```
1: fazer  $f_{\text{best}} = +\infty$ 
2: for  $i = 1, \dots, N$  do
3:   gerar  $\mathbf{x}_i$  uniforme em  $D$ 
4:   calcular  $f(\mathbf{x}_i)$ 
5:   if  $f(\mathbf{x}_i) < f_{\text{best}}$  then fazer  $\mathbf{x}_{\text{best}} = \mathbf{x}_i$ ;  $f_{\text{best}} = f(\mathbf{x}_i)$ 
6:   end if
7: end for
8: return  $\mathbf{x}_{\text{best}}$ 
```

$$n_f = N$$

- Sem direção, sem passo, sem memória: os pontos são independentes (e, por isso, paralelizáveis).
- Usa só valores de f (ordem 0) e um domínio D limitado.
- Garantia: sob hipóteses usuais (domínio compacto, amostragem com suporte total, f contínua), o melhor valor aproxima o ótimo *global* em probabilidade quando $N \rightarrow \infty$ — devagar.

Quão devagar? É a pergunta seguinte.

Análise experimental: quão depressa se aproxima?

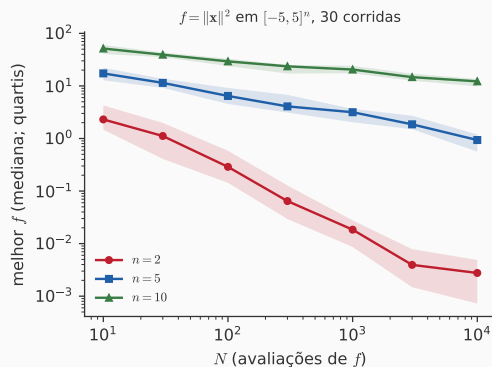
$f = \|\mathbf{x}\|^2$ em $[-5, 5]^2$, 30 corridas (método estocástico: **mediana e quartis**):

N	melhor f	$\ \mathbf{x}_{\text{best}}\ $ [Q1; Q3]
10	2,3	1,5 [1,2; 2,1]
100	0,29	0,54 [0,38; 0,76]
1 000	0,018	0,14 [0,09; 0,17]
10 000	0,0028	0,053 [0,027; 0,070]

Para $n = 2$, multiplicar N por 10 divide tipicamente a distância por $\sqrt{10} \approx 3$. Em geral:

$\|\mathbf{x}_{\text{best}} - \mathbf{x}^*\| \sim N^{-1/n}$ (aqui $f = \|\mathbf{x}\|^2$, logo $f_{\text{best}} \sim N^{-2/n}$). Para $\|\mathbf{x}\| \approx 10^{-3}$: $N \sim 10^7$.

Neste problema separável, uma pesquisa unidimensional eficiente em cada coordenada chega a essa precisão com algumas dezenas de avaliações.



Linha: mediana; faixa: quartis. Quanto maior n , mais devagar.

A maldição da dimensão

Porque é que a distância decresce como $N^{-1/n}$? A probabilidade de um ponto uniforme em $[-5, 5]^n$ cair na bola de raio ε à volta do mínimo é

$$p(\varepsilon) = \frac{\text{vol}(B_\varepsilon)}{10^n} \propto \left(\frac{\varepsilon}{10}\right)^n,$$

logo precisamos de $N \sim 1/p \propto (10/\varepsilon)^n$ amostras para esperar acertar uma vez. Ignorando a constante geométrica $\text{vol}(B_1)$, é a escala que interessa, não o número exato.

Pergunta: se, ignorando a constante, em 2D a escala é 10^4 (com $\varepsilon = 0,1$), o que acontece em 10D?

A maldição da dimensão

Porque é que a distância decresce como $N^{-1/n}$? A probabilidade de um ponto uniforme em $[-5, 5]^n$ cair na bola de raio ε à volta do mínimo é

$$p(\varepsilon) = \frac{\text{vol}(B_\varepsilon)}{10^n} \propto \left(\frac{\varepsilon}{10}\right)^n,$$

logo precisamos de $N \sim 1/p \propto (10/\varepsilon)^n$ amostras para esperar acertar uma vez. Ignorando a constante geométrica $\text{vol}(B_1)$, é a **escala** que interessa, não o número exato.

Pergunta: se, ignorando a constante, em 2D a escala é 10^4 (com $\varepsilon = 0,1$), o que acontece em 10D?

	$n = 2$	$n = 5$	$n = 10$
escala $(10/\varepsilon)^n$, $\varepsilon = 0,1$	10^4	10^{10}	10^{20}

Com a constante: $1/p \approx 3 \cdot 10^3$, $2 \cdot 10^9$, $4 \cdot 10^{19}$ — os números mudam, a escala não.

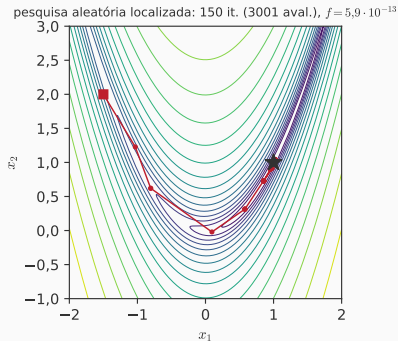
Na figura anterior vê-se o mesmo: para $n = 10$, 10^4 avaliações ainda dão $f \approx 12$ (mediana). A pesquisa aleatória pura só é viável em dimensão muito baixa ou como **primeira fase** de outro método.

Porque não concentrar a pesquisa? A variante localizada

Se amostrar sempre o domínio todo é tão ineficiente, amostramos **à volta do melhor ponto**, em $\mathbf{x} + r[-1, 1]^n$, e encolhemos r quando não há progresso:

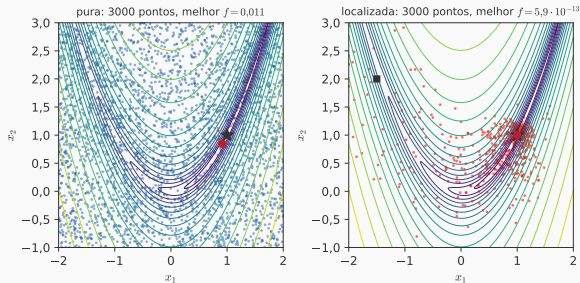
- 1: fazer $\mathbf{x} = \mathbf{x}_0$; $r = r_0$; $k = 0$
- 2: **loop**
- 3: gerar m pontos $\mathbf{x} + \xi$, ξ uniforme em $[-r, r]^n$
- 4: **if** o melhor deles melhora $f(\mathbf{x})$ **then** aceitar: fazer $\mathbf{x}$ igual a esse ponto
- 5: **else** fazer $r = \gamma r$ ($\gamma \approx 0,9$)
- 6: **end if**
- 7: fazer $k = k + 1$; parar se $r \leq \varepsilon_x$ ou $k \geq k_{\max}$
- 8: **end loop**

Ganha **localização**: explora em torno do melhor ponto $\mathbf{x}$, com uma escala de pesquisa r . Perde a propriedade global e torna-se uma pesquisa local: pode aproximar um mínimo local, mas o resultado depende do ponto inicial, das amostras e dos parâmetros.



Rosenbrock de $(-1,5; 2)$, $m = 20$, $r_0 = 1$, $\gamma = 0,9$: uma corrida (150 iterações, $n_f = 3001$, com $f(\mathbf{x}_0)$) chega ao vale e segue-o até $(1, 1)$. Em 30 corridas (sementes 0–29), atinge $f < 10^{-4}$ com $n_f = 975$ [867; 1052] (mediana [Q1; Q3]).

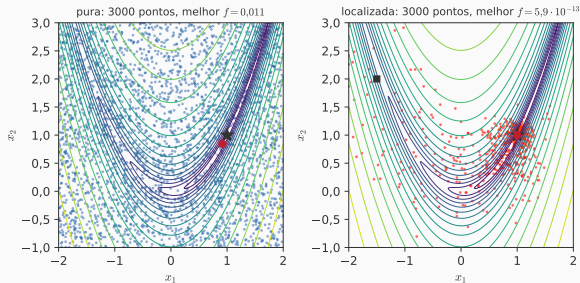
Pura ou localizada? O mesmo orçamento no Rosenbrock



Uma corrida de cada, $n_f = 3000$ (pura) e 3001 (localizada, com $f(\mathbf{x}_0)$); à direita, a cor indica a ordem das amostras (vermelho: primeiras).

Neste problema, com estes parâmetros.

Pura ou localizada? O mesmo orçamento no Rosenbrock



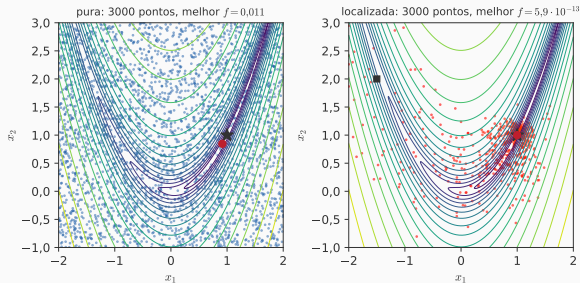
Qual usa melhor as avaliações?

A localizada: concentra-se e acompanha o vale. Em 30 corridas, melhor f , mediana [Q1; Q3]: 0,014 [0,008; 0,022] (pura) vs. $2 \cdot 10^{-12}$ [$5 \cdot 10^{-13}$; $8 \cdot 10^{-12}$] (localizada).

Uma corrida de cada, $n_f = 3000$ (pura) e 3001 (localizada, com $f(\mathbf{x}_0)$); à direita, a cor indica a ordem das amostras (vermelho: primeiras).

Neste problema, com estes parâmetros.

Pura ou localizada? O mesmo orçamento no Rosenbrock



Uma corrida de cada, $n_f = 3000$ (pura) e 3001 (localizada, com $f(\mathbf{x}_0)$); à direita, a cor indica a ordem das amostras (vermelho: primeiras).

Neste problema, com estes parâmetros.

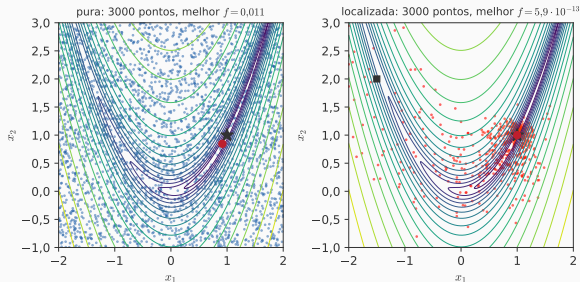
Qual usa melhor as avaliações?

A localizada: concentra-se e acompanha o vale. Em 30 corridas, melhor f , mediana [Q1; Q3]: 0,014 [0,008; 0,022] (pura) vs. $2 \cdot 10^{-12}$ [$5 \cdot 10^{-13}$; $8 \cdot 10^{-12}$] (localizada).

E qual pode escapar para outra região?

A pura: cobre o domínio todo; a localizada só vê a vizinhança do ponto onde está.

Pura ou localizada? O mesmo orçamento no Rosenbrock



Uma corrida de cada, $n_f = 3000$ (pura) e 3001 (localizada, com $f(\mathbf{x}_0)$); à direita, a cor indica a ordem das amostras (vermelho: primeiras).

Neste problema, com estes parâmetros.

Qual usa melhor as avaliações?

A localizada: concentra-se e acompanha o vale. Em 30 corridas, melhor f , mediana [Q1; Q3]: 0,014 [0,008; 0,022] (pura) vs. $2 \cdot 10^{-12}$ [$5 \cdot 10^{-13}$; $8 \cdot 10^{-12}$] (localizada).

E qual pode escapar para outra região?

A pura: cobre o domínio todo; a localizada só vê a vizinhança do ponto onde está.

exploração global



exploração local eficiente

Um compromisso que reaparece no cap. 5.

No computador: o algoritmo inteiro

```
rng(s); % semente
fbest = inf;
for i = 1:N
    x = a + (b-a).*rand(1,n); % em [a,b]
    fx = f(x);
    if fx < fbest
        xbest = x;
        fbest = fx;
    end
end
```

$$n_f = N$$

As funções completas (RandomSearch, LocalRandomSearch, com info.nfev e info.nhit, o custo até atingir o alvo usado na comparação final do capítulo) estão no código complementar da UC.

Algoritmo estocástico $\Rightarrow$ várias corridas

rng(s) fixa a semente: cada corrida é reprodutível, mas o resultado é uma **variável aleatória**. Reporta-se a **mediana e os quartis** de 20–30 corridas (sementes diferentes), nunca uma corrida isolada.

É a regra que volta no *simulated annealing* (5.2), nos algoritmos genéticos (5.3) e no NSGA-II (6.6).

O QUE DEVE FICAR — 3.2.1

1. Pesquisa aleatória pura: gerar → avaliar → guardar o melhor.
2. Usa apenas valores de f (ordem 0) e custa $n_f = N$ avaliações.
3. Com amostragem uniforme, $\|\mathbf{x}_{\text{best}} - \mathbf{x}^*\| \sim N^{-1/n}$: sofre fortemente com a dimensão.
4. Localizar a pesquisa melhora a eficiência local, mas perde a natureza global e passa a depender do ponto inicial, das amostras e dos parâmetros.
5. É estocástico: não se avalia uma corrida isolada — várias sementes, mediana e quartis.

Um método sem derivadas só é interessante se usar as avaliações melhor do que pontos ao acaso — com o mesmo n_f , no mesmo problema.

A seguir (3.2.2): espalhámos pontos ao acaso. E se quisermos cobrir o domínio de forma sistemática? — a pesquisa em grelha.

Referências Lista completa: *Bibliografia da UC*, no fim do PDF da UC.

- Conn, A. R., Scheinberg, K., Vicente, L. N., *Introduction to Derivative-Free Optimization*, SIAM (MPS-SIAM Series on Optimization 8), 2009 — cap. 1.
- Arora, J. S., *Introduction to Optimum Design*, 3.^a ed., Academic Press, 2011 — cap. 18.

Recursos

- Código complementar: `RandomSearch`, `LocalRandomSearch` (`info.nfev`, `info.nhit`). MATLAB: `rng(s)`.